

Feuille de travaux pratiques n° 3

Parallélisation de calcul d'image

— CORRECTION —

Code de départ associé : <https://github.com/DistributedComputing/TP-mandelbrot>.

Le but de ce travail est de paralléliser un programme affichant une représentation graphique de l'ensemble de Mandelbrot. Une grande portion du code a déjà été faite pour vous. Le code de départ est disponible sur le dépôt GitHub du TP.

1 Architecture du programme

Le programme suit une architecture en deux composants :

- un client graphique (fenêtre Swing) chargé de l'affichage,
- un serveur de calcul responsable du rendu de l'image.

Le serveur graphique implémente l'interface `Drawer`, qui contient une méthode `draw(Graphics2D)` appelée à chaque fois que la fenêtre client doit être rafraîchie. Le serveur doit alors dessiner, sur l'objet `Graphics2D` fourni, une image qui dépend de trois paramètres connus du serveur.

- La fonction mathématique à tracer `function`, de type `DoubleBinaryOperator`, représentant les fonctions à deux arguments de type `double` (des coordonnées du plan), et retournant un `double` (la valeur de la fonction en ce point). Dans ce TP, nous utiliserons l'implémentation `Mandelbrot`.
- Un objet de type `ColorPalette` qui spécifie la couleur des pixels à afficher à l'écran en fonction de la valeur retournée par la fonction.
- Une collection d'objets de type `Area` qui font le lien entre les zones du plan à dessiner et leur position sur l'écran. On fournit une collection d'objets `Area` au serveur, plutôt qu'un seul grand rectangle, pour pouvoir paralléliser le calcul de l'image plus facilement : chaque objet `Area` pourra être traitée par une tâche différente.

Enfin, le client, dont le comportement est défini dans la classe `Client`, gère la fenêtre et délègue le dessin au serveur. Il mesure également le temps nécessaire à l'affichage complet de l'image ; vous n'avez donc pas besoin d'ajouter vos propres chronomètres. Le code du client n'est pas à modifier.

La fenêtre rappelle périodiquement la méthode `Drawer.draw(Graphics2D)` (environ dix fois par seconde). À chaque appel, le `canvas` fourni est un nouveau canevas qui doit être entièrement redessiné. Il est donc important de distinguer deux étapes effectuées par le serveur graphique :

- Le *calcul* de l'image, réalisé par un appel à `Area.getImage(function, palette)`, ne doit être effectué qu'une seule fois par zone, faute de quoi l'image serait recalculée à chaque rafraîchissement ;
- Le *dessin* de l'image déjà calculée, réalisé par un appel à `area.drawImage(graphics, image)`, doit être répété à chaque appel de `draw`, car le canevas change à chaque rafraîchissement.

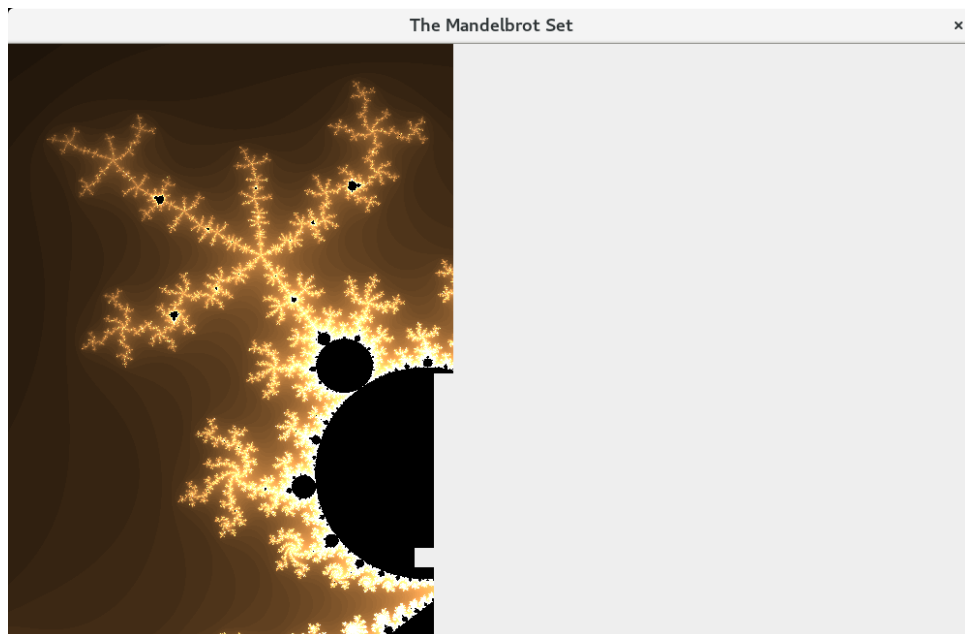


FIGURE 1 – Capture d’écran en cours d’exécution, avec 10 threads.

2 Travail à réaliser

Le programme fourni inclut une classe `SequentialDrawer`, qui contient une implémentation séquentielle et bloquante de l’interface `Drawer`. Cette classe calcule à chaque rafraîchissement de la fenêtre l’image complète de l’ensemble de Mandelbrot, en appelant successivement `Area.getImage(function, palette)` pour chaque zone. Le résultat est ensuite immédiatement affiché sur le canevas graphique par un appel à `area.drawImage(graphics, image)`. Comme l’image est recalculée à chaque appel de `draw`, cette implémentation est très lente, mais elle constitue une base simple à partir de laquelle vous allez construire vos propres versions parallèles. Votre but est de réimplémenter le serveur en suivant différentes stratégies. Vous ne devez pas modifier cette classe, mais en créer de nouvelles (par exemple `AsyncDrawer`, `PerAreaDrawer`, `PoolDrawer`) qui implémentent la même interface. La figure 1 montre une capture d’écran obtenue pendant l’exécution du programme après parallélisation.

Mesurez le temps d’exécution des différentes solutions, et tracez le graphe du temps nécessaire pour obtenir l’image complète en fonction du nombre de threads (faites varier avec un pas de un entre un seul thread au total jusqu’à environ 20 threads, puis vous pouvez espacer les mesures jusqu’à un thread par objet `Area`).

Vous pouvez modifier les paramètres de la méthode `Main.main()` pour que les temps de calcul soient mieux adaptés à votre machine. Un temps autour de 30 secondes devrait convenir pour observer des effets intéressants.

1. Vous devez proposer une nouvelle implémentation de l’interface `Drawer` dans laquelle le calcul de l’image doit être effectué une seule fois et dans un thread séparé. La méthode `draw` doit retourner `false` tant que l’image n’est pas prête, puis afficher l’image et retourner `true` ensuite.
2. Même question, mais en parallélisant le calcul de l’image en lançant un thread par objet `Area`.
3. Même question, mais en parallélisant le calcul de l’image en lançant un total de n threads qui se répartissent les objets `Area`.